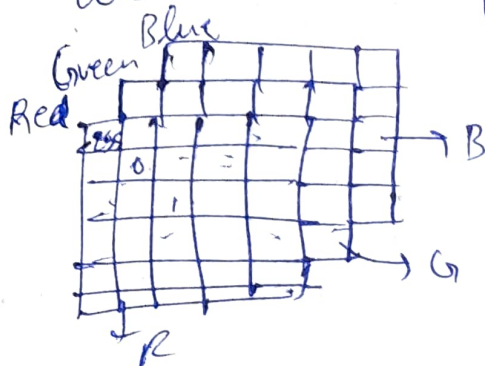


Binary Classification

Ex. Classifying a photo to whether it is a cat or not cat

We take the pixels of the photo



it is stored as 3 matrices each n representing R, G, B values of each pixel

To work on this image, we turn it into a feature vector like this:

$$X = \begin{pmatrix} 255 \\ 231 \\ \vdots \\ 255 \\ 128 \\ \vdots \\ 138 \\ 210 \end{pmatrix} \begin{matrix} \left. \begin{matrix} \text{Red values} \\ \text{Green value} \\ \text{Blue values} \end{matrix} \right\} \end{matrix} \quad 64 \times 64 \times 3 = 12288 = n$$

Now, ~~we~~ we use X to predict 1 or 0 $\rightarrow$ cat $\rightarrow$ not cat

• Important Notation $\Rightarrow$

- A single training example is represented as: (x, y) where x is a n dimensional feature vector and y is 1 or 0 saying if its a cat or not

Then m training examples look like: $\{(x^1, y^1), (x^2, y^2), (x^3, y^3), \dots, (x^m, y^m)\}$

m or m_{train}

Test samples: m_{test}

Now, we can represent the training set more compactly by using another matrix X such that:

$$X = \begin{bmatrix} x^1 & x^2 & \dots & x^m \\ \vdots & \vdots & & \vdots \end{bmatrix} \begin{matrix} \uparrow \\ n.\text{length} \\ \downarrow \end{matrix}$$

$X \in \mathbb{R}^{n \times m}$
or
 $X.\text{shape} = (n, m)$

In certain places, they might take x^{iT} but $\leftarrow$ it is easier in practice to use above.

$\begin{bmatrix} x^{1T} & \dots & x^{iT} & \dots & x^{mT} \\ \vdots & & \vdots & & \vdots \end{bmatrix}$
like this

We can also take another matrix Y to represent all the outputs:

$$Y = [y^1, y^2, \dots, y^m] \quad Y \in \mathbb{R}^{1 \times m}$$

$Y.\text{shape} = (1, m)$